BCSE307P — COMPILER DESIGN

MatrixDSL

Project Explainer & Phase 2 Summary
for Team and Faculty Review

Project ID	**A3**
Team Number	**13**
Purpose	**Plain-language walkthrough of the project, Phase 2 additions, and individual contributions**

TEAM MEMBERS

Name	Register Number	Role
Vinay A	24BCB0131	Lexer, Parser, AST
Mudpe Parth Tulsidas	24BDS0353	Semantic Analysis, Symbol Table
Kandi Jeevitesh Reddy	24BCE0350	LLVM IR, Tensor IR, Optimization
Shreyas Mandem	24BCE0381	MDT ISA, Backend, Simulator

Repository · github.com/shreyasmandem/MatrixDSL

Part 1 — What MatrixDSL is

The problem in one sentence

Almost all of modern AI is matrix multiplication, but ordinary programming languages hide that fact from the compiler, so it can neither catch a dimension mistake before the program runs nor use the specialised matrix-multiply hardware that real AI chips have.

Why that's a real problem

If you write a matrix multiply in a general-purpose language, it just looks like three nested loops:

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j];
```

The programmer knows this is a matrix multiplication. The compiler does not — it only sees arithmetic on numbers. Two consequences follow. First, a dimension mistake (multiplying a 4×3 matrix by a 7×5 one) isn't caught until the program actually runs, if it's caught at all. Second, modern AI chips (NVIDIA's tensor cores, Google's TPU) have a single hardware instruction that multiplies a whole small matrix at once — but a compiler can only use it if it *recognises* those three loops as a matrix multiply, and that recognition is fragile and easily defeated by the compiler's own optimizations.

Our answer

Make matrices a first-class part of the language, so nothing has to be recovered later:

```
matrix A[4][4];
matrix B[4][4];
C = matmul(A, B);
```

Now the compiler knows exactly what's happening, at every stage, because nothing was ever thrown away.

The two things we built

1. **MatrixDSL** — the small language above.
2. **MDT (MatrixDSL Target)** — a processor we designed ourselves, on paper. It has registers that hold an *entire matrix* (not just one number), and a single instruction, `MATMUL`, that multiplies two whole matrices at once. MDT doesn't physically exist, so we also built **a simulator** — a program that pretends to be the chip, so we can actually run and check our output.

The compiler pipeline

```
MatrixDSL source
      |
   Lexer            (Vinay)   chop text into tokens
      |
   Parser           (Vinay)   check grammar, build a tree (the AST)
      |
Semantic Analysis    (Parth)  check the maths makes sense, catch shape errors
      |
LLVM IR Generation   (Kandi)  translate to LLVM's standard middle language
      |
LLVM Optimization    (Kandi)  hand off to LLVM's own optimizer (we don't write this)
      |
MDT Backend         (Shreyas) translate into our own chip's instructions
      |
MDT Assembly + Simulator (Shreyas)  actually run it and check the answer
```

Review 1 built the language, the ISA, and a working simulator that could run 17 basic instructions. Phase 2 — this review — is about pushing the whole pipeline toward something that actually looks like a modern AI compiler.

Part 2 — What we did for Phase 2

The honest starting point: reviewing what we had

Before adding anything, we looked hard at Review 1's design and asked what was still missing to call this an "AI compiler" with a straight face. Three gaps stood out:

- **Only single 2-D matrices.** Real AI workloads process *batches* — many matrices at once (a batch of images, a batch of attention heads). We had no way to say "this is a group of 8 matrices," only one at a time.
- **No memory hierarchy.** Real chips have a small, fast on-chip memory (a "scratchpad") sitting between slow main memory and the compute registers. Our simulator only had one flat memory — no way to show *why* a smart compiler moving data around actually helps.
- **No operator fusion.** In real compilers, if you compute a matrix multiply and immediately apply an activation function to it, a good compiler fuses those into a single hardware operation instead of two. We had no such thing.

We also decided, just as deliberately, what **not** to add — full support for arbitrary-dimension tensors, dynamic (runtime-decided) shapes, and a proper multi-core, truly concurrent memory system were all

explicitly ruled out as too large for four students in the time we have. Scoping down correctly is as much a design decision as scoping something up.

What we actually added

1. A batch dimension. `matrix A[4][4]` still works exactly as before, but now you can also write `tensor X[8][4][4]` — a batch of 8 independent 4×4 matrices. This one small addition is what makes it possible to express real AI workloads like a transformer layer or an attention block.

2. `softmax`, a fourth built-in operation, alongside `matmul`, `transpose`, and `relu` — the normalisation step that turns raw numbers into something that behaves like a probability distribution, and a core building block of every modern attention mechanism.

3. A Tensor IR — a new "thinking layer" in the middle of the compiler. We looked seriously at whether to adopt MLIR, the industrial-strength framework that real systems like Google's and Meta's AI compilers use. We decided against it — not because the idea is wrong, but because adopting the whole framework is a multi-month undertaking on its own, on top of everything else four people already have to build in one semester. Instead we built a small, hand-rolled version of the *same idea*: a layer where the compiler can recognise patterns like "a matrix multiply immediately followed by an activation" and rewrite them into something faster, before handing the result down to LLVM.

4. Seven new instructions for our chip, MDT. The most important one is `MATMULRELU` — a single instruction that does a matrix multiply *and* applies the activation function in one step. This is a real technique used by production AI compilers (it's literally how NVIDIA and Google's compilers speed up real neural networks) — we're doing a small, honest version of the same idea. The others add a "remember what you last computed" instruction (`MATMULACC`), reduced-precision number handling for saving memory bandwidth, and instructions needed to compute `softmax`.

5. A scratchpad memory — a second, faster memory tier. We gave our simulated chip a small, fast on-chip memory in addition to its main memory, and made moving data in and out of it cost fewer "cycles" than going to main memory. This isn't just cosmetic — it lets us actually *measure*, with real numbers, that a program which reuses data through the scratchpad runs measurably cheaper than one that keeps re-reading from main memory every time. That's the entire point of a memory hierarchy in real hardware, shown at a scale we can build and check ourselves.

6. A performance model. Not a full hardware simulation — we're upfront that it isn't cycle-accurate — but a simple, honest way to add up how "expensive" a program is, and to show the benefit of good memory use in concrete numbers rather than just a claim.

Proving it actually works

Every one of the additions above was built, compiled, and tested — not just written and hoped to be correct. When a real C++ compiler became available partway through this phase, the very first thing we did was build and run everything already written for Review 1, and it caught two real bugs that had been sitting in the code, undetected, since the previous review. That's mentioned here on purpose: it's the reason we trust the numbers in this report, because we watched the tools actually catch our own mistakes before they could reach a reviewer.

Part 3 — What each of us did

Vinay A — Lexer, Parser, AST

I own the very front of the compiler: turning the text of a MatrixDSL program into a structured tree the rest of the compiler can work with.

For Phase 2, I extended the grammar to understand the two new pieces of syntax — the batch dimension and `softmax` — and, just as importantly, I finished the two pieces that were left as interfaces only at Review 1: a full working parser, and the code that walks the resulting tree and prints it out for debugging. Both are built and tested — 28 out of 28 tests pass, including tests I wrote specifically to check that things like operator precedence (`A + B * C` correctly treats `*` as tighter-binding than `+`) and left-to-right associativity are genuinely correct, not just assumed.

Mudpe Parth Tulsidas — Semantic Analysis, Symbol Table

My job is deciding whether a syntactically valid program actually *makes sense* — do the matrix shapes actually line up? This is the single strongest argument for the whole project: a general-purpose compiler simply cannot check this, because it doesn't know the program is doing matrix algebra in the first place.

For Phase 2, I extended every shape-checking rule to understand the new batch dimension — two batched tensors can only be combined if their batch counts match, or one of them is a single (non-batched) matrix that gets reused across the batch. I also finished the piece that was left as an interface-only promise at Review 1: the actual code that walks the parsed program and applies every shape rule to it, reporting a clear, specific error the moment something doesn't line up.

Kandi Jeevitesh Reddy — LLVM IR Generation, Tensor IR, Optimization

I sit in the middle of the compiler, between the front end and the parts that generate real machine code. My biggest Phase 2 contribution is the new Tensor IR layer described above — I built the actual data structure that represents a matrix program at this "thinking" level, and the pass that scans it for the pattern "a matrix multiply immediately followed by an activation function" and rewrites it into a single fused operation. I made sure this pass is careful, not greedy — it only fuses when the intermediate result isn't needed anywhere else in the program, because fusing something that's used twice would silently produce the wrong answer. I proved this works correctly with both a positive test (the fusion happens when it should) and a negative test (it correctly refuses to fuse when it shouldn't).

Shreyas Mandem — MDT ISA, Backend, Memory Hierarchy, Simulator

I own everything below the optimizer: the design of our chip (MDT), and the simulator that lets us actually run programs on it since no real hardware exists. For Phase 2 I designed and implemented all seven new instructions, added the second, faster memory tier (the scratchpad) and the addressing scheme that lets the compiler choose between it and main memory, and built the performance model that turns a program into real, comparable numbers.

The most important thing I can say about this work is that all of it is *verified*, not just written. When a compiler finally became available, I found and fixed two real bugs in code that had shipped, unverified, at

Review 1 — and while building the new performance model, I hand-traced my own first design and found it was mathematically wrong before it ever shipped, which is exactly the kind of mistake that's easy to make and easy to miss if you don't check your own arithmetic against the actual behaviour of the code you wrote.

Closing note for the review

Nothing in this document is a claim we can't back up on request: every number, every test result, and every design decision above has a corresponding file, test, or commit behind it. We're happy to open any of them and walk through exactly how we arrived at it.